Document	P3396R1
Date	2024-11-20
Reply To	Lewis Baker <lewissbaker@gmail.com>
Audience	LWG

std::execution wording fixes

Abstract

When P2300R10 was merged into the working draft, a subsequent review of the github issue tracker discovered several outstanding issues relating to the wording that was in P2300R10 which were not addressed during the LWG review.

Now that the P2300 wording has been merged into the working draft, addressing these issues needs to be done either via LWG issues or a paper. It was suggested that an omnibus paper containing the outstanding issues may be more efficiently processed by LWG than filing separate issues for each. This is that paper.

Wording Issues

Note to editor: All editing instructions shown below are to be applied to the working draft.

#1 The preconditions on `run_loop::run()` are too strict

Source: https://github.com/cplusplus/sender-receiver/issues/280

In [exec.run.loop.members] p5 the precondition for the run() member function states:

> *Preconditions:* `state` is *starting*

However, one of the common use-cases is for algorithms like `sync_wait()` [exec.sync.wait] which connects/starts an operation that calls `run_loop::finish()` upon completion, and once `start()` has returned, calls `run_loop::run()` to drive the event-loop until the operation finishes. In the case that the operation completes synchronously (or possibly just quickly on

another thread), this can mean that the `run_loop::finish()` function can potentially occur before the call to `run_loop::run()`.

But as the call to `run_loop::finish()` transitions the state from *starting* to *finishing* this would mean that the subsequent call to `run_loop::run()` would fail to satisfy its preconditions, which require that the state is *starting*.

As this is a use-case that needs to work - i.e. a prior call to `run_loop::finish()` should unblock a subsequent call to `run_loop::run()` - we need to modify the precondition for run_loop::run() to allow the state to be either *starting* or *finishing*. However, to allow us to continue to distinguish the case where `run()` has already been called and has returned, we need to add an additional *finished* state. Also, to prevent `finish()` from being called again after `run()` has returned and thus making it valid to then call `run_loop::run()` again, we need to add a pre-condition to `finish()` that checks that `finish()` has not been called before.

Proposed Resolution

Modify [exec.run.loop.general] p2 as follows:

> A `run_loop` instance has an associated *count* that corresponds to the number of work items that are in its queue.
> Additionally, a `run_loop` instance has an associated state that can be one of *starting*, *running*, ~~or~~ *finishing*, or *finished*.

Modify [exec.run.loop.members] p1 as follows:

> *run-loop-opstate-base* pop-front*`();`
>
> *Effects*: Blocks ([defns.block]) until one of the following conditions is `true`:
>
> - *count* is `0` and *state* is *finishing*, in which case *pop-front* sets *state* to *finished* and returns `nullptr`; or
> - *count* is greater than `0`, in which case an item is removed from the front of the queue, *count* is decremented by `1`, and the removed item is returned.

Modify [exec.run.loop.members] p5/6 as follows:

> `void run();`
>
> *Preconditions*: *state* is one of *starting* or *finishing*.
>
> *Effects*: If *state* is *starting*, ~~S~~sets the *state* to *running*, otherwise leaves *state* unchanged. Then, equivalent to:
>
> ```
> while (auto* op = pop-front()) {
>   op->execute();
> }
> ```

Modify [exec.run.loop.members] p8/9 as follows:

> `void finish();`
>
> *Preconditions:* *state* is one of *starting* or *running*.
> *Effects*: Changes *state* to *finishing*.

Synchronization: `finish` synchronizes with the *`pop-front`* operation that returns `nullptr`.

#2 `noexcept` clause of *basic-state* constructor is incomplete

Source: https://github.com/cplusplus/sender-receiver/issues/279

In [exec.snd.expos] p28, it specifies the expression in the `noexcept` clause of the constructor of *`basic-state`* as:

```
is_nothrow_move_constructible_v<Rcvr> &&
nothrow-callable<decltype(impls-for<tag_of_t<Sndr>>::get-
state), Sndr, Rcvr&>
```

However, this doesn't take into account the fact that the return-type of *`get-state`* might return a reference which is used to initialize a data-member of the decayed return type of *`get-state`* and thus might call a constructor.

The state member of *`basic-state`* has type *`state-type`*`<Sndr, Rcvr>` where *`state-type`* is defined as:

```
template<class Sndr, class Rcvr> // exposition only
using state-type = decay_t<call-result-t<
   decltype(impls-for<tag_of_t<Sndr>>::get-state), Sndr,
Rcvr&>>;
```

The default get-state implementation is defined as equivalent to:

```
[]<class Sndr, class Rcvr>(Sndr&& sndr, Rcvr& rcvr)
noexcept -> decltype(auto) {
  auto& [_, data, ...child] = sndr;
  return std::forward_like<Sndr>(data);
}
```

Which for senders like those returned from `execution::then(src, f)`, results in passing a reference to the *`data`* member initialized from `f`. If a const-lvalue sender is passed to `execution::connect()`, then a const-lvalue-reference to the *`data`* member will be returned from get-state and will be used to initialize the *`state`* member of *`basic-state`*, resulting in a call to the copy-constructor for the function-object.

This can make the *`basic-state`* constructor potentially throwing, even though the `noexcept` specification of the constructor evaluates to `noexcept(true)`.

For example:

```
std::string s = "a really long string that doesn't
trigger small object optimization";
const auto sndr =
std::execution::then(std::execution::just(),
                                        [s=std::move(s)]
noexcept { return std::move(s); });
auto op = sndr.connect(some_receiver{});
```

To fix this, we need to add an extra expression to the expression of the noexcept specifier that takes the potential call to a constructor here into account. Note that it also needs to handle the case where the data-member is initialized directly through copy-elision (i.e. where the return-type and decayed return-type are

identical and thus copy-elision is guaranteed and an additional call to the constructor will not occur.

Proposed Resolution

Modify [exec.snd.expos] p28 as follows:

> The expression in the `noexcept` clause of the constructor of *`basic-state`* is:
>
> ```
> is_nothrow_move_constructible_v<Rcvr> &&
> nothrow-callable<decltype(impls-
> for<tag_of_t<Sndr>>::get-state), Sndr, Rcvr&> &&
> (same_as<state-type<Sndr, Rcvr>, get-state-result> ||
> is_nothrow_constructible_v<state-type<Sndr, Rcvr>,
> get-state-result>)
> ```
>
> Where *`get-state-result`* is *`call-result-t`*`<decltype(`*`impls-for`*`<tag_of_t<Sndr>>::`*`get-state`*`), Sndr, Rcvr&>`.

#3 Definition of an async operation's environment's ownership seems incorrect

Source: https://github.com/cplusplus/sender-receiver/issues/271

In [exec.async.ops] p4 it states:

> An asynchronous operation has an associated environment.
> An *environment* is a queryable object ([exec.queryable]) representing the execution-time properties of the operation's caller.
> The caller of an asynchronous operation is its parent operation or the function that created it.
> An asynchronous operation's operation state owns the operation's environment.

The last sentence "An asynchronous operation's operation state owns the operation's environment" does not seem to reflect the actual nature of the relationship.

An operation's environment is provided to it by its caller via the receiver, and while the child operation state takes ownership of the receiver, usually storing it as a data-member, the environment returned by the receiver's get_env() member function typically has reference semantics - with the actual state of the environment stored across several parent operation-states. Destroying the operation-state does not necessarily destroy the environment or the resources it provides access to.

Further, the environment returned by the receiver's get_env() function is often constructed on-demand, so saying that the receiver "owns" the environment - and by proxy, since the operation-state owns the receiver, that the operation-state owns the environment - seems like a stretch.

The actual relationship is more one of the caller provides a receiver during connect() which provides access to the environment for the duration of the

operation. It is not valid to access the environment after the operation has completed as the caller may release resources referenced by the environment upon completion of a child operation.

I propose we either just don't say anything about the ownership of the environment, or alternatively say something about the validity of the environment.

Proposed Resolution

Modify [exec.async.ops] p4 as follows:

> An asynchronous operation has an associated environment.
> An *environment* is a queryable object ([exec.queryable]) representing the execution-time properties of the operation's caller.
> The caller of an asynchronous operation is its parent operation or the function that created it.
> ~~An asynchronous operation's operation state owns the operation's environment.~~

#4 scheduler semantic requirements imply swapability but concept does not require it

Source: https://github.com/cplusplus/sender-receiver/issues/270

The scheduler concept is defined in [exec.sched] as follows:

```
template<class Sch>
concept scheduler =
          derived_from<typename
remove_cvref_t<Sch>::scheduler_concept, scheduler_t> &&
          queryable<Sch> &&
          requires(Sch&& sch) {
            { schedule(std::forward<Sch>(sch)) } ->
sender;
            { auto(get_completion_scheduler<set_value_t>(
                get_env(schedule(std::forward<Sch>
(sch))))) }
                  -> same_as<remove_cvref_t<Sch>>;
          } &&
          equality_comparable<remove_cvref_t<Sch>> &&
          copy_constructible<remove_cvref_t<Sch>>;
```

However, [exec.sched] p3 also states:

> None of a scheduler's copy constructor, destructor, equality comparison, or `swap` member functions shall exit via an exception.
> None of these member functions, nor a scheduler type's `schedule` function, shall introduce data races as a result of potentially concurrent ([intro.races]) invocations of those functions from different threads.

The wording for p3 mentions a swap() member function, but the concept does not require a `swap` member-function, or the `swappable` concept.

Should the `scheduler` concept also require `swappable<Sch>`?

Also, the scheduler concept does not require move-assignability of schedulers, so the default `std::swap()` implementation is not necessarily going to work for types that minimally implement the concept.

Should the concept also require the type to be `movable`? Or possibly even `copyable`?
Note that `copyable` requires `movable` which requires `swappable`.

Proposed Resolution

Modify the scheduler concept in [exec.sched] as follows:

```
template<class Sch>
concept scheduler =
   derived_from<typename
remove_cvref_t<Sch>::scheduler_concept, scheduler_t> &&
   queryable<Sch> &&
   requires(Sch&& sch) {
     { schedule(std::forward<Sch>(sch)) } -> sender;
     { auto(get_completion_scheduler<set_value_t>(
              get_env(schedule(std::forward<Sch>(sch)))))
}
        -> same_as<remove_cvref_t<Sch>>;
   } &&
   equality_comparable<remove_cvref_t<Sch>> &&
   ~~copy_constructible<remove_cvref_t<Sch>>;~~
   copyable<remove_cvref_t<Sch>>;
```

We may also want to adjust p3 to refer to "`swap` operation", or "`swap` function", rather than "`swap` member-function" as the latter is not guaranteed to be used by a call to `std::ranges::swap()`.

Modify [exec.sched] p3 as follows:

> ~~None of a scheduler's copy constructor, destructor,, equality comparison, or `swap` member functions~~ No operation required by `copyable<remove_cvref_t<Sch>>` and `equality_comparable<remove_cvref_t<Sch>>` shall exit via an exception.
> None of these ~~member functions~~operations, nor a scheduler type's `schedule` function, shall introduce data races as a result of potentially concurrent ([intro.races]) invocations of those ~~functions~~operations from different threads.

NOTE to editor: The proposed resolution here also addresses #8.

#5 *operation-state-task* exposition-only type does not need a move constructor

Source: https://github.com/cplusplus/sender-receiver/issues/236

Section [exec.connect] p4 defines the *operation-state-task* exposition-only type as having a move-constructor which can transfer ownership of the `coroutine_handle` to another instance.

However, as the *operation-state-task* class is modeling the `operation_state` concept, which does not require the object to be move-constructible, the move-constructor defined in this class is unnecessary.

I suggest we instead mark the move-constructor as deleted. This can also simplify the destructor, which no longer needs to handle a moved-from state.

Proposed Resolution

Modify [exec.connect] p4 as follows:

> Let *operation-state-task* be the following exposition-only class:

```
namespace std::execution {
  struct operation-state-task {
    using operation_state_concept = operation_state_t;
    using promise_type = connect-awaitable-promise;

    explicit operation-state-task(coroutine_handle<> h)
noexcept : coro(h) {}
    operation-state-task(operation-state-task&& o)
noexcept= delete;
    : coro(exchange(o.coro, {})) {}
    ~operation-state-task() { if (coro) coro.destroy();
}

    void start() & noexcept {
      coro.resume();
    }

  private:
    coroutine_handle<> coro;
              // exposition only
  };
}
```

#6 [exec.general] Wording for AS-EXCEPT-PTR should use 'Precondition:' instead of 'Mandates:' for runtime properties

Source: https://github.com/cplusplus/sender-receiver/issues/234

At [exec.general] p8 The definition of the exposition-only `AS-EXCEPT-PTR` has a case for when the error is already an `exception_ptr` that has the following requirement:

> *Mandates*: `err != exception_ptr()` is `true`.

"Mandates" clauses are usually for things which are ill-formed and can be detected at compile-time.
However this is a runtime property and so should probably be a *Precondition*: instead.

Proposed Resolution

Change "*Mandates*" in [exec.general] p8.1.sentence-2, as indicated:

> ~~*Mandates*~~*Precondition*: `!err` ~~`!= exception_ptr()`~~ is ~~`true`~~`false`.

#7 [exec.schedule.from] Potential access to destroyed state in *`impls-for`*`::`*`complete`*

Source: https://github.com/cplusplus/sender-receiver/issues/233

The current specification of `schedule_from` algorithm has the following definition for *`impls-for`*`<schedule_from_t>::`*`complete`*:

```
[]<class Tag, class... Args>(auto, auto& state, auto&
rcvr, Tag, Args&&... args) noexcept
   -> void {
  using result_t = decayed-tuple<Tag, Args...>;
  constexpr bool nothrow =
is_nothrow_constructible_v<result_t, Tag, Args...>;

  TRY-EVAL(std::move(rcvr), [&]() noexcept(nothrow) {
        state.async-result.template emplace<result_t>
(Tag(), std::forward<Args>(args)...);
  }());

  if (state.async-result.valueless_by_exception())
        return;
  if (state.async-result.index() == 0)
        return;
```

```
    start(state.op-state);
  };
```

The call to *TRY-EVAL* here will invoke `set_error()` on the receiver if the construction of `result_t` throws an exception.
As the call to `set_error()` can potentially end up destroying the operation-state, the subsequent access of `state.async-result.valueless_by_exception()` is potentially accessing a dangling reference to `state`.

Instead, we need to have this function `return;` after `set_error()` is called.
This may mean we need to directly define the expansion of *TRY-EVAL* instead of using the exposition-only macro.

Proposed Resolution

Modify [exec.schedule.from] p11 as follows:

> The member *impls-for*<schedule_from_t>::*complete* is initialized with a callable object equivalent to the following lambda:

```
[]<class Tag, class... Args>(auto, auto& state, auto&
rcvr, Tag, Args&&... args) noexcept
    -> void {
  using result_t = decayed-tuple<Tag, Args...>;
  constexpr bool nothrow =
is_nothrow_constructible_v<result_t, Tag, Args...>;

  TRY-EVAL(rcvr, [&]() noexcept(nothrow) {
  try {
        state.async-result.template emplace<result_t>
(Tag(), std::forward<Args>(args)...);
 } catch (...) {
    if constexpr (!nothrow) {
      set_error(std::move(rcvr), current_exception());
      return;
    }
  }
  }());

  if (state.async-result.valueless_by_exception())
        return;
  if (state.async-result.index() == 0)
        return;

  start(state.op-state);
};
```

#8 scheduler concept should require that move-constructor does not exit with an exception

Source: https://github.com/cplusplus/sender-receiver/issues/116

[exec.sched] p3 states:

> None of a scheduler's copy constructor, destructor, equality comparison, or swap member functions shall exit via an exception.
> None of these member functions, nor a scheduler type's `schedule` function, shall introduce data races as a result of potentially concurrent ([intro.races]) invocations of those functions from different threads.

The scheduler concept requires the type to be std::copy_constructible, which subsumes std::move_constructible. The above paragraph requires the scheduler's copy-constructor to not exit via an exception, but does not require the move-constructor to not exit via an exception.

Further, the std::copy_constructible concept subsumes std::destructible, which already requires that the destructor is declared noexcept. Thus the above paragraph is redundantly requiring the destructor to not exit with an exception, which is already enforced by requiring the destructor to be noexcept.

Note that this has potential interactions with issue #4 listed above in this document, which suggests changing the std::copy_constructible requirement to std::copyable. If this is the case, then we may also want to explicitly list the copy-assignment and move-assignment operations as also not throwing exceptions.

Proposed Resolution

Addressed by the resolution of #4.

#9 [exec.bulk] wording should indicate that `f` is called with `i` = [0, …, shape-1]

Source: https://github.com/cplusplus/sender-receiver/issues/102

The current wording for [exec.bulk] p6.1 is:

> - on a value completion operation, invokes `f(i, args...)` for every `i` of type `Shape` from `0` to `shape`, where `args` is a pack of lvalue subexpressions referring to the value completion result datums of the input sender, and

It is not clear from this wording that the intent is to have f invoked with i = 0, i = 1, … up to i = shape-1, but not invoked with i = shape.

Proposed Resolution

Modify [exec.bulk] p6.1 as follows:

> - on a value completion operation, invokes `f(i, args...)` for every `i` of type `Shape` ~~from `0` to `shape`~~in [`0`, `shape`), where `args` is a pack of lvalue subexpressions referring to the value completion result datums of the input sender, and

#10 The use of JOIN-ENV leads to inefficient implementations of composed environments

Source: https://github.com/cplusplus/sender-receiver/issues/229

The current wording for the exposition-only *write-env()* function ([exec.snd.expos]), as well as `let_value`, `let_error`, `let_stopped` ([exec.let]), `when_all` and `when_all_with_variant` ([exec.when.all]) defines the `get_env()` operation on receivers passed to child operations in terms of the exposition-only `JOIN-ENV` helper.

The *JOIN-ENV* helper [exec.snd.expos] p4 is defined as follows:

> For two queryable objects `env1` and `env2`, a query object `q`, and a pack of subexpressions `as`, *JOIN-ENV*`(env1, env2)` is an expression `env3` whose type satisfies `queryable` such that `env3.query(q, as…)` is expression-equivalent to:
>
> - `env.query(q, as…)` if that expression is well-formed,
> - otherwise, `env2.query(q, as…)` if that expression is well-formed,
> - otherwise, `env3.query(q, as…)` is ill-formed.

And then we have usage in, e.g. [exec.when.all] p6 that states:

> The member *impls-for*`<when_all_t>::`*get-env* is initialized with a callable object equivalent to the following lambda expression:
>
> ```
> []<class State, class Rcvr>(auto&&, State& state, const
> Receiver& rcvr) noexcept {
>  return JOIN-ENV(
>    MAKE-ENV(get_stop_token,
> state.stop_src.get_token()), get_env(rcvr));
> }
> ```

The problem with the current formulation is that this forces implementations to evaluate the environments eagerly. As `env1` and `env2` are specified to be 'objects' this means that it must evaluate the argument expressions once to produce the object, and since this might be a temporary, the object must be stored in the returned object. However, if we apply this same approach recursively over a tree of such senders, then we end up with every call to `get_env()` at the leaf level potentially copying the entire set of environment values of every ancestor environment.

For example, if we apply `when_all()` to a nesting depth of 5, then at the leaf operation level, the call to the leaf receiver's `get_env()` will return an environment that contains a stop-token plus the parent environment. The parent-environment contains a stop-token plus its parent environment, which contains a stop-token and its parent environment, etc. until the returned environment contains 5 separate stop-token objects, only one of which is returned if someone queries for the stop-token, and none of which are used if the caller is querying for something else.

Ideally, the places where we use JOIN-ENV would permit implementations to return a `queryable` object that only calls `get_env()` on the parent receiver's

environment if the query is not satisfied immediately by the parent operation state.

We may be able to better express this by instead describing the behavior of queries on the queryable returned from get_env() rather than describing `get_env()` in code.

For example, by saying it returns an object, `env`, of unspecified type modeling `queryable` such that:

- `env.query(get_stop_token)` is expression-equivalent to '`state.stop_src.get_token()`'
- given a query object, `q`, of decayed type other than `get_stop_token_t`, `env.query(q)` is expression-equivalent to `get_env(rcvr).query(q)`

Proposed Resolution

Modify [exec.when.all] p6 as follows:

> The member *`impls-for`*`<when_all_t>::`*`get-env`* is initialized with a callable object equivalent to the following lambda expression:
>
> ```
> []<class State, class Rcvr>(auto&&, State& state, const
> Receiver& rcvr) noexcept {
>  return /* see below */JOIN-ENV(
>    MAKE-ENV(get_stop_token,
> state.stop_src.get_token()), get_env(rcvr));
> }
> ```
>
> Returns an object `e` such that:
>
> - `decltype(e)` models `queryable`; and
> - `e.query(get_stop_token)` is expression-equivalent to `state.stop_src.get_token()`; and
> - given a query object, `q`, with type other than *cv* `stop_token_t`, `e.query(q)` is expression-equivalent to `get_env(rcvr).query(q)`

Modify [exec.let] p6 as follows:

> Let *`receiver2`* denote the following exposition-only class template:
>
> ```
> namespace std::execution {
>   template<class Rcvr, class Env>
>   struct receiver2 {
>     using receiver_concept = receiver_t;
>
>     template<class... Args>
>     void set_value(Args&&... args) && noexcept {
>       execution::set_value(std::move(rcvr),
> std::forward<Args>(args)...);
>     }
>
>     template<class Error>
>     void set_error(Error&& err) && noexcept {
>       execution::set_error(std::move(rcvr),
> std::forward<Error>(err));
>     }
>
>     void set_stopped() && noexcept {
>       execution::set_stopped(std::move(rcvr));
> ```

```
    }

    decltype(auto) get_env() const noexcept {
      return /* see below */JOIN-ENV(env, FWD-
ENV(execution::get_env(rcvr)));
    }

    Rcvr& rcvr;                          // exposition only
    Env env;                             // exposition only
  };
}
```

Where *receiver2*`::get_env()` returns an object, `e`, such that:

- `decltype(e)` models `queryable`; and
- Given a query object, `q`, the expression `e.query(q)` is expression-equivalent to *env*`.query(q)` if that expression is valid, otherwise `e.query(q)` is expression-equivalent to `get_env(`*rcvr*`).query(q)`.

Modify [exec.snd.expos] p43 as follows:

Remarks: The exposition-only class template *impls-for* ([exec.snd.general]) is specialized for *write-env-t* as follows:

```
template<>
struct impls-for<write-env-t> : default-impls {
  static constexpr auto get-env =
    [](auto, const auto& state, const auto& rcvr)
noexcept {
      return /* see below */JOIN-ENV(state,
get_env(rcvr));
    };
};
```

Where *impls-for*`<`*write-env-t*`>::`*get-env* returns an object, e, such that:

- `decltype(e)` models `queryable`; and
- Given a query object, `q`, the expression `e.query(q)` is expression-equivalent to `state.query(q)` if that expression is valid, otherwise, `e.query(q)` is expression-equivalent to `get_env(rcvr).query(q)`.